Human Verification: `Containers.ZappaSzep`

Zappa–Szép Products: Cocycle Axioms ⇔ Associativity

Machine-generated from `lean/ZappaSzep.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.ZappaSzep` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file formalises the *pairwise Zappa–Szép criterion*: given two monoids L and G with a left action $\triangleright : G \times L \to L$ and a restriction $\triangleleft : G \times L \to G$, the binary operation $(a, g) \cdot (b, h) = (a \cdot (g \triangleright b),\ (g \triangleleft b) \cdot h)$ on $L \times G$ is associative *if and only if* four cocycle axioms `ZS1`–`ZS4` hold (given unit compatibility).

Contents

1 Overview **1**

2 Algebraic data **2**
2.1 Structure: `MatchedPair` 2

3 The Zappa–Szép product **3**
3.1 Definition: `MatchedPair.mul` 3

4 The four cocycle axioms **3**
4.1 Definition: `MatchedPair.ZS1` 3
4.2 Definition: `MatchedPair.ZS2` 3
4.3 Definition: `MatchedPair.ZS3` 4
4.4 Definition: `MatchedPair.ZS4` 4
4.5 Definition: `MatchedPair.Assoc` 4

5 Main theorems **4**
5.1 Theorem: `MatchedPair.assoc_of_zs` 4
5.2 Theorem: `MatchedPair.zs_of_assoc` 5
5.3 Theorem: `MatchedPair.zs_iff_assoc` 5

6 Sanity-check example **5**
6.1 Definition: `natDirect` 5

1 Overview

The Zappa–Szép product is a non-abelian generalisation of the semidirect product. Where a semidirect product requires one group to act on the other, the Zappa–Szép product requires a

matched pair: two *mutual* actions satisfying four compatibility axioms.

The main theorem establishes the equivalence: `ZS1` $\wedge$ `ZS2` $\wedge$ `ZS3` $\wedge$ `ZS4` $\Leftrightarrow$ Assoc. The forward direction ($\Rightarrow$) uses no unit laws; the converse ($\Leftarrow$) recovers each axiom by specialising associativity at unit elements.

2 Algebraic data

2.1 Structure: `MatchedPair`

Definition 2.1 (Matched pair of monoids)**.** A **matched pair** consists of:

- two monoids $(L, \cdot_L, 1_L)$ and $(G, \cdot_G, 1_G)$,
- a left action $\triangleright : G \times L \to L$ ("G acts on L from the left"), and
- a restriction $\triangleleft : G \times L \to G$ ("L acts on G from the right"),

together with four **unit compatibility conditions**:

$$\begin{aligned} &\text{(U1a)} \quad 1_G \triangleright a = a, && \text{(U1b)} \quad 1_G \triangleleft a = 1_G, \\ &\text{(U2a)} \quad g \triangleright 1_L = 1_L, && \text{(U2b)} \quad g \triangleleft 1_L = g. \end{aligned}$$

No cocycle axiom is assumed: `ZS1`–`ZS4` are the *content* of the main theorem and are stated separately.

Lean.

```
structure MatchedPair where
  L : Type
  G : Type
  mulL : L → L → L
  oneL : L
  mulG : G → G → G
  oneG : G
  mulL_assoc : ∀ a b c, mulL (mulL a b) c = mulL a (mulL b c)
  oneL_mul   : ∀ a, mulL oneL a = a
  mul_oneL   : ∀ a, mulL a oneL = a
  mulG_assoc : ∀ g h k, mulG (mulG g h) k = mulG g (mulG h k)
  oneG_mul   : ∀ g, mulG oneG g = g
  mul_oneG   : ∀ g, mulG g oneG = g
  lact       : G → L → L   -- notation lact g a = g ⟩ a
  ract       : G → L → G   -- notation ract g a = g ⟨ a
  lact_oneG  : ∀ a, lact oneG a = a
  ract_oneG  : ∀ a, ract oneG a = oneG
  lact_oneL  : ∀ g, lact g oneL = oneL
  ract_oneL  : ∀ g, ract g oneL = g
```

Haskell.

```
data MatchedPair l g = MatchedPair
  { mulL    :: l → l → l
  , oneL    :: l
  , mulG    :: g → g → g
  , oneG    :: g
  , lact    :: g → l → l   -- g |> a
  , ract    :: g → l → g   -- g <| a
  -- plus proof obligations for associativity, units, and U1a/b, U2a/b
  }
```

Fidelity note. In Lean, the monoid laws and unit-compatibility conditions are bundled as *fields* of the structure (proofs are data at `Prop` universe). The Haskell rendering cannot enforce them in the type; they become documentation or runtime-checked invariants.

3 The Zappa–Szép product

3.1 Definition: `MatchedPair.mul`

Definition 3.1 (Zappa–Szép product operation)**.** Given a matched pair P, define a binary operation on $L \times G$ by:

$$(a, g) \cdot (b, h) \;=\; (a \cdot_L (g \triangleright b),\;\; (g \triangleleft b) \cdot_G h).$$

Lean.

```
def MatchedPair.mul (x y : P.L * P.G) : P.L * P.G :=
  (P.mulL x.1 (P.lact x.2 y.1), P.mulG (P.ract x.2 y.1) y.2)
```

Haskell.

```
zsMul :: MatchedPair l g → (l, g) → (l, g) → (l, g)
zsMul p (a, g) (b, h) =
  (mulL p a (lact p g b), mulG p (ract p g b) h)
```

4 The four cocycle axioms

4.1 Definition: `MatchedPair.ZS1`

Definition 4.1 (ZS1: left action over product)**.**

$$g \triangleright (a \cdot_L b) \;=\; (g \triangleright a) \cdot_L ((g \triangleleft a) \triangleright b).$$

The left action distributes over the product in L, twisted by the restriction.

Lean.

```
def MatchedPair.ZS1 : Prop :=
  ∀ (g : P.G) (a b : P.L),
    P.lact g (P.mulL a b) = P.mulL (P.lact g a) (P.lact (P.ract g a) b
      )
```

Haskell.

```
zs1 :: MatchedPair l g → Bool  -- for all g a b:
-- lact g (mulL a b) == mulL (lact g a) (lact (ract g a) b)
```

4.2 Definition: `MatchedPair.ZS2`

Definition 4.2 (ZS2: left action of G)**.**

$$(g \cdot_G h) \triangleright a \;=\; g \triangleright (h \triangleright a).$$

The left action is an action of G (i.e. a monoid homomorphism $G \to \mathrm{End}(L)$).

Lean.

```
def MatchedPair.ZS2 : Prop :=
  ∀ (g h : P.G) (a : P.L),
    P.lact (P.mulG g h) a = P.lact g (P.lact h a)
```

Haskell.

```
-- zs2: lact (mulG g h) a == lact g (lact h a)
```

4.3 Definition: `MatchedPair.ZS3`

Definition 4.3 (ZS3: restriction over product in G)**.**

$$(g \cdot_G h) \triangleleft a \;=\; (g \triangleleft (h \triangleright a)) \cdot_G (h \triangleleft a).$$

Lean.

```
def MatchedPair.ZS3 : Prop :=
  ∀ (g h : P.G) (a : P.L),
    P.ract (P.mulG g h) a = P.mulG (P.ract g (P.lact h a)) (P.ract h a
       )
```

Haskell.

```
-- zs3: ract (mulG g h) a == mulG (ract g (lact h a)) (ract h a)
```

4.4 Definition: `MatchedPair.ZS4`

Definition 4.4 (ZS4: right action of L)**.**

$$g \triangleleft (a \cdot_L b) \;=\; (g \triangleleft a) \triangleleft b.$$

The restriction is a right action of L on G.

Lean.

```
def MatchedPair.ZS4 : Prop :=
  ∀ (g : P.G) (a b : P.L),
    P.ract g (P.mulL a b) = P.ract (P.ract g a) b
```

Haskell.

```
-- zs4: ract g (mulL a b) == ract (ract g a) b
```

4.5 Definition: `MatchedPair.Assoc`

Definition 4.5 (Associativity of the Zappa–Szép product)**.** The operation `mul` on $L \times G$ is associative:

$$\forall\, x, y, z : L \times G, \quad (x \cdot y) \cdot z = x \cdot (y \cdot z).$$

Lean.

```
def MatchedPair.Assoc : Prop :=
  ∀ x y z : P.L * P.G, P.mul (P.mul x y) z = P.mul x (P.mul y z)
```

Haskell.

```
assocProp :: MatchedPair l g → Bool  -- for all x y z:
-- zsMul p (zsMul p x y) z == zsMul p x (zsMul p y z)
```

5 Main theorems

5.1 Theorem: `MatchedPair.assoc_of_zs`

Theorem 5.1 (Forward direction: ZS1–ZS4 imply associativity)**.** Suppose P is a matched pair and the four cocycle axioms hold. Then the Zappa–Szép product is associative.

Proof sketch: the L-coordinate follows from ZS1 (re-associating the action over a product) and ZS2 (the action law); the G-coordinate from ZS3 and ZS4. No unit law is used.

Lean.

```
theorem MatchedPair.assoc_of_zs
    (h1 : P.ZS1) (h2 : P.ZS2) (h3 : P.ZS3) (h4 : P.ZS4) :
    P.Assoc
```

Haskell.
```
assocOfZS :: ZS1 p → ZS2 p → ZS3 p → ZS4 p → Assoc p
-- Proof: equational, L-coord from ZS1+ZS2, G-coord from ZS3+ZS4.
```

5.2 Theorem: `MatchedPair.zs_of_assoc`

Theorem 5.2 (Converse direction: associativity implies ZS1–ZS4)**.** If the Zappa–Szép product is associative, then all four cocycle axioms hold.

Proof sketch: each axiom is recovered by specialising associativity at unit elements. ZS1 and ZS2 come from the L-coordinate of specific instances; ZS3 and ZS4 from the G-coordinate. The unit-compatibility fields of `MatchedPair` are exactly what is needed to collapse the instances.

Lean.
```
theorem MatchedPair.zs_of_assoc (H : P.Assoc) :
    P.ZS1 /\ P.ZS2 /\ P.ZS3 /\ P.ZS4
```

Haskell.
```
zsOfAssoc :: Assoc p → (ZS1 p, ZS2 p, ZS3 p, ZS4 p)
-- Proof: specialise associativity at unit elements.
```

5.3 Theorem: `MatchedPair.zs_iff_assoc`

Theorem 5.3 (Main theorem: equivalence)**.** For any matched pair P:

$$(\mathrm{ZS1} \wedge \mathrm{ZS2} \wedge \mathrm{ZS3} \wedge \mathrm{ZS4}) \iff \mathrm{Assoc}.$$

Lean.
```
theorem MatchedPair.zs_iff_assoc :
    (P.ZS1 /\ P.ZS2 /\ P.ZS3 /\ P.ZS4) <→ P.Assoc :=
  ⟨fun ⟨h1, h2, h3, h4⟩ ⇒ P.assoc_of_zs h1 h2 h3 h4, P.zs_of_assoc⟩
```

Haskell.
```
zsIffAssoc :: (ZS1 p, ZS2 p, ZS3 p, ZS4 p) <→ Assoc p
zsIffAssoc = (uncurry4 assocOfZS, zsOfAssoc)
```

6 Sanity-check example

6.1 Definition: `natDirect`

Definition 6.1 (Direct product of $(\mathbb{N}, +, 0)$ with itself)**.** The **trivial matched pair**: $L = G = \mathbb{N}$, multiplication $+$, unit 0, left action $g \triangleright a = a$ (trivial), restriction $g \triangleleft a = g$ (identity).

This is the direct product: $(a, g)\cdot(b, h) = (a{+}b, g{+}h)$. The four cocycle axioms hold trivially, and `zs_iff_assoc` recovers associativity of $(\mathbb{N}, +)$ — a sanity check that `MatchedPair` is inhabited.

Lean.
```
def natDirect : MatchedPair where
  L := Nat;  G := Nat
  mulL := Nat.add;  oneL := 0
  mulG := Nat.add;  oneG := 0
  mulL_assoc := fun a b c ⇒ Nat.add_assoc a b c
  oneL_mul   := fun a ⇒ Nat.zero_add a
  mul_oneL   := fun a ⇒ Nat.add_zero a
  mulG_assoc := fun a b c ⇒ Nat.add_assoc a b c
  oneG_mul   := fun a ⇒ Nat.zero_add a
  mul_oneG   := fun a ⇒ Nat.add_zero a
  lact := fun _ a ⇒ a
```

```
  ract := fun g _ ⇒ g
  lact_oneG := fun _ ⇒ rfl;  ract_oneG := fun _ ⇒ rfl
  lact_oneL := fun _ ⇒ rfl;  ract_oneL := fun _ ⇒ rfl
```

Haskell.

```
natDirect :: MatchedPair Int Int
natDirect = MatchedPair
  { mulL = (+), oneL = 0
  , mulG = (+), oneG = 0
  , lact = \_  a → a    -- trivial action
  , ract = \g  _ → g    -- identity restriction
  }
```